DOCUMENTO DE VISÃO DE PROJETO



JoStore

FATEC
Faculdade de Tecnologia de Mogi das Cruzes
Tecnologia em Análise e Desenvolvimento de Sistemas | Tecnologia em Agronegócio | Tecnologia em Recursos Humanos



Histórico de Versões

Data	Versão	Descrição	Autor	Revisor
07/09/2026	1.0	Modelagem e desenvolvimento	Andrey Marques	-
24/09/2026	1.1	Revisão geral do DVP e atualização das visões técnicas	Andrey Marques	-

Cliente Rodrigo Rocha - Laboratório de Engenharia de Software

Documento Documento de Visão de Projeto: *JoStore*

Data 07 de setembro de 2026

Autor **Andrey Marques**
andreymmarques6@gmail.com

Página de Assinaturas

Revisado e Aprovado por: _____

07/09/26



Índice

OBJETIVO **3**
ESCOPO 4
REFERÊNCIAS **4**
NECESSIDADES DE NEGÓCIO **4**
OBJETIVO DO PROJETO **4**
DECLARAÇÃO PRELIMINAR DE ESCOPO **5**
DESCRIÇÃO 5
PRODUTOS A SEREM ENTREGUES 5
REQUISITOS 5
REQUISITOS FUNCIONAIS 5
REQUISITOS NÃO FUNCIONAIS 6
REGRAS DE NEGÓCIO 6
PREMISSAS **6**
INFLUÊNCIA DAS PARTES INTERESSADAS **7**
REPRESENTAÇÃO ARQUITETURAL **7**
RESTRIÇÕES ARQUITETURAIS 8
OBJETIVOS E RESTRIÇÕES ARQUITETURAIS 8
VISÃO DE CASOS DE USO **9**
DIAGRAMA DE CASOS DE USO DE PEDIDOS 10
DESCRIÇÃO DO CASO DE USO DE PEDIDOS 10
VISÃO DE LÓGICA **11**
CAMADA DE APRESENTAÇÃO 12
CAMADA DE NEGÓCIO 12
PACOTE CONTROLLER 13
PACOTE MODEL 14
CAMADA DE PERSISTÊNCIA 14
REALIZAÇÃO DOS CASOS DE USO SIGNIFICATIVOS 15
VISÃO DE IMPLANTAÇÃO **16**
VISÃO DE IMPLEMENTAÇÃO **17**
VISÃO DE DADOS **17**
TAMANHO E PERFORMANCE **19**
QUALIDADE **19**
CRONOGRAMA MACRO **19**
REFERÊNCIAS **21**

OBJETIVO

Este documento tem como objetivo formalizar o início do projeto JoStore, apresentando uma visão geral e arquitetônica da solução a ser desenvolvida ao longo da disciplina de Laboratório de Engenharia de Software.



O JoStore tem como propósito introduzir uma loja física, até então sem presença tecnológica, à gestão de vendas online, oferecendo um canal de e-commerce que amplia seu alcance e moderniza sua operação de vendas.

Escopo

Este documento aborda a visão inicial do sistema JoStore, delimitando seus objetivos, necessidades de negócio e uma visão preliminar de arquitetura. O detalhamento dos requisitos funcionais e não funcionais, das regras de negócio, da modelagem de dados e da arquitetura completa do sistema serão tratados em documentos complementares a serem elaborados ao longo da disciplina.

REFERÊNCIAS

Este documento foi elaborado com base nas aulas e orientações da disciplina de Laboratório de Engenharia de Software, ministrada pelo Prof. Rodrigo Rocha, e em conversas com a proprietária da loja física de roupas que serviu de base para o projeto, atuando como cliente/stakeholder do sistema. Este documento influencia diretamente o Documento de Requisitos, bem como o planejamento e o cronograma das próximas fases do projeto.

NECESSIDADES DE NEGÓCIO

Atualmente, a loja realiza suas vendas exclusivamente de forma presencial, complementadas por contatos informais via WhatsApp e Instagram. Esse modelo limita o alcance do negócio a clientes que já conhecem a loja ou estão fisicamente próximos, dificultando a captação de novos clientes e a competitividade frente a concorrentes que já possuem presença consolidada no comércio eletrônico. Sem uma solução de vendas online estruturada, a loja tende a perder espaço de mercado, restringindo seu crescimento e sua capacidade de atender à demanda de clientes que preferem comprar pela internet.

OBJETIVO DO PROJETO

O objetivo do JoStore é oferecer uma solução de vendas online que permita à loja:

- Disponibilizar um catálogo de produtos (roupas) para venda pela web;
- Permitir que o cliente monte um carrinho de compras e finalize pedidos (checkout);
- Gerenciar cadastro e autenticação de clientes;
- Controlar e acompanhar pedidos de venda;
- Gerenciar o cadastro e o estoque de produtos;
- Controlar as informações financeiras relacionadas às vendas.

Com isso, espera-se ampliar o alcance de mercado da loja e permitir que ela compita em pé de igualdade com concorrentes que já vendem online.



DECLARAÇÃO PRELIMINAR DE ESCOPO

Esta seção apresenta uma declaração preliminar do escopo do projeto JoStore, contemplando uma visão inicial dos produtos a serem entregues e dos principais requisitos identificados nesta fase. O detalhamento completo dos requisitos, das regras de negócio e das funcionalidades será tratado no Documento de Requisitos.

Descrição

A JoStore é um sistema de e-commerce voltado à venda de roupas, desenvolvido para atender a dois perfis de usuário: o cliente final, que navega pelo catálogo de produtos, monta seu carrinho de compras e finaliza pedidos online; e a administradora da loja, responsável por gerenciar o catálogo de produtos, acompanhar os pedidos realizados e controlar informações financeiras relacionadas às vendas. O sistema tem como propósito levar a operação de uma loja física de roupas para o ambiente digital, ampliando seu alcance de vendas.

Produtos a serem entregues

Ao final desta etapa do projeto, serão entregues os seguintes produtos:

- *Sistema JoStore completo, com front-end e back-end integrados e funcionando com dados reais (persistidos em banco de dados), evoluindo a partir da versão inicial do front-end apresentada com dados mockados;*
- *Documento de Visão de Projeto (DVP);*
- *Documentação técnica complementar prevista para as próximas etapas, incluindo diagramas de classes de domínio e modelagem de banco de dados;*
- *Possível disponibilização do sistema em ambiente de hospedagem, a ser definida ao final do projeto.*

Requisitos

Esta seção apresenta uma visão preliminar e de alto nível dos principais requisitos identificados nesta fase inicial do projeto JoStore, servindo como ponto de partida para o Documento de Requisitos, que detalhará fluxos, exceções e regras de negócio de forma completa.

Requisitos Funcionais

O sistema deve ser capaz de:

- *Cadastrar, consultar, alterar e inativar um cliente;*
- *Cadastrar, consultar, alterar e remover cartões de crédito e endereços de um cliente;*
- *Permitir que o cliente gerencie seu carrinho de compras;*
- *Exibir um catálogo de produtos (roupas) disponíveis para venda;*
- *Cadastrar, consultar, alterar e controlar o estoque de produtos;*
- *Gerenciar o fluxo do pedido, desde a sua realização até a entrega ou a solicitação de troca;*
- *Registrar as informações financeiras relacionadas às vendas realizadas.*



Requisitos Não Funcionais

O sistema deve atender aos seguintes requisitos não funcionais:

- O back-end deve ser desenvolvido na linguagem Java, utilizando o framework Spring Boot;
- O front-end deve ser desenvolvido em React com TypeScript;
- O sistema deve utilizar o banco de dados PostgreSQL para persistência de dados;
- A arquitetura do sistema deve seguir o padrão MVC (Model-View-Controller);
- O sistema deve ser compatível com as versões atuais dos navegadores Google Chrome, Microsoft Edge e Mozilla Firefox.

Regras de Negócio

Além dos marcos apresentados, foram identificadas as seguintes regras de negócio nesta fase inicial do projeto:

- Um cliente não pode se cadastrar duas vezes com o mesmo CPF/e-mail;
- Um cliente inativado não pode realizar novos pedidos, mas seu histórico é mantido;
- Um cliente só pode finalizar uma compra se estiver autenticado (logado);
- Um produto sem estoque disponível não pode ser adicionado ao carrinho;
- A quantidade em estoque deve ser reduzida automaticamente após a confirmação do pedido;
- Um produto não pode ter preço ou estoque negativo;
- O carrinho não pode ser finalizado (checkout) se estiver vazio;
- O cliente deve possuir ao menos um endereço cadastrado para finalizar a compra;
- O cliente deve possuir ao menos um cartão de crédito cadastrado para concluir o pedido;
- Um pedido só pode ser cancelado antes de ser despachado/enviado;
- O valor total do pedido deve ser calculado automaticamente com base nos itens do carrinho, sem edição manual;
- Um pagamento recusado não deve gerar confirmação do pedido nem baixa de estoque;
- O status do pedido deve seguir um fluxo definido, permitindo consultar cliente e pedidos, e alterar o status segundo as transições: EM ABERTO → EM PROCESSAMENTO → PAGAMENTO REALIZADO → EM TRÂNSITO → ENTREGUE; e, para trocas: TROCA SOLICITADA → TROCA ACEITA/NEGADA, ITEM ENVIADO → ITEM RECEBIDO → TROCA PROCESSADA.

PREMISSAS

Para o desenvolvimento do projeto JoStore, assumem-se as seguintes premissas:

- *O projeto será orientado pelo professor Rodrigo Rocha, da disciplina de Laboratório de Engenharia de Software;*
- *O desenvolvimento é conduzido individualmente pelo autor deste documento;*
- *A proprietária da loja física estará disponível para validar requisitos e fornecer informações sobre o negócio ao longo do projeto;*
- *O ambiente de desenvolvimento (Java/Spring Boot, React/TypeScript, PostgreSQL) está disponível e configurado para uso imediato;*
- *Os prazos e marcos de entrega seguirão o cronograma definido pela disciplina.*



-

INFLUÊNCIA DAS PARTES INTERESSADAS

Interessado	Papel	Interesse/Influência
Prof. Rodrigo Rocha	Orientador da disciplina	Aderência aos requisitos acadêmicos da disciplina e cumprimento dos prazos/marcos estabelecidos
Proprietária da loja	Cliente real do projeto	Que o sistema atenda às necessidades reais do negócio: visibilidade online, facilidade de gestão de vendas, produtos e clientes
Andrey Marques Alves Menezes	Desenvolvedor/autor	Entregar um sistema funcional e bem arquitetado, aplicando os conceitos de engenharia de software aprendidos na disciplina

REPRESENTAÇÃO ARQUITETURAL

O JoStore adota uma arquitetura cliente-servidor, estruturada em camadas. A camada de apresentação é implementada como uma Single Page Application (SPA) em React com TypeScript. A camada de negócio é implementada em Java com Spring Boot, organizada nos seguintes componentes: Controllers, responsáveis por expor os endpoints da API REST e receber as requisições; Services, responsáveis pela lógica de negócio; Repositories, responsáveis pelo acesso a dados via Spring Data JPA; Models/Entities, que representam as entidades de domínio; DTOs (Data Transfer Objects), utilizados para o transporte de dados entre as camadas e a API; e uma camada de Exceptions, responsável pelo tratamento centralizado de erros. A camada de persistência utiliza o banco de dados PostgreSQL.

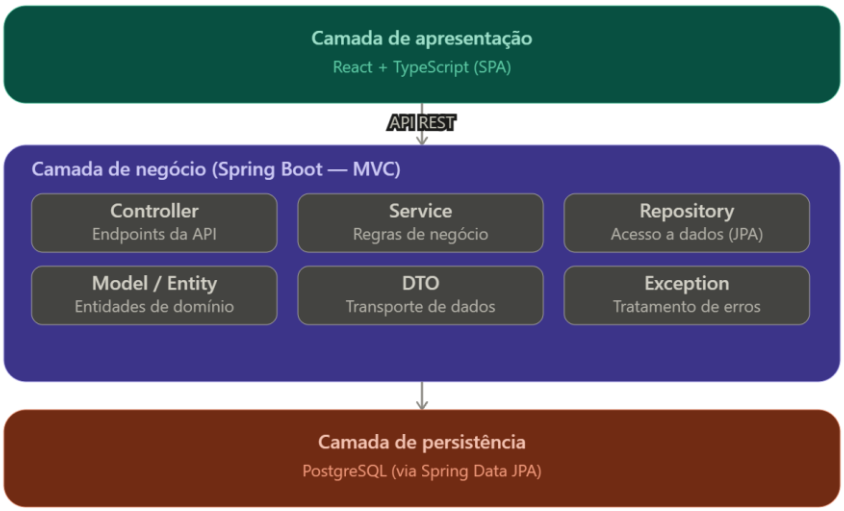


Figura 1 - Modelo Arquitetural

A Figura acima ilustra a arquitetura em camadas adotada pelo JoStore. A camada de apresentação, no topo, é responsável pela interface com o usuário e é implementada como uma Single Page Application (SPA) em React com TypeScript, comunicando-se com o back-end através de requisições a uma API REST. A camada de negócio, ao centro, é implementada em Java com Spring Boot, seguindo o padrão MVC, e organiza-se internamente em seis componentes: as Controllers, responsáveis por expor os endpoints da API e receber as requisições do front-end; os Services, responsáveis por concentrar as regras de negócio da aplicação; os Repositories, responsáveis pelo acesso aos dados via Spring Data JPA; os Models/Entities, que representam as entidades de domínio do sistema; os DTOs (Data Transfer Objects), utilizados para transportar dados entre as camadas sem expor diretamente as entidades; e a camada de Exceptions, responsável pelo tratamento centralizado de erros da aplicação. Por fim, a camada de persistência, na base, é responsável pelo armazenamento dos dados do sistema, utilizando o banco de dados PostgreSQL.

Restrições Arquiteturais

O projeto adota as seguintes restrições arquiteturais:

- *Back-end desenvolvido em Java, utilizando o framework Spring Boot;*
- *Persistência de dados via Spring Data JPA, com banco de dados PostgreSQL;*
- *Front-end desenvolvido em React com TypeScript;*
- *Comunicação entre front-end e back-end via API REST;*
- *Organização do back-end nas camadas Controller, Service, Repository, Model, DTO e Exception.*

Objetivos e Restrições Arquiteturais



O JoStore não realiza integração com gateways de pagamento externos; o processamento de pagamento é simulado internamente pelo próprio sistema, sem comunicação com serviços de terceiros. Como restrição de segurança, o sistema implementa a criptografia das senhas dos usuários antes de seu armazenamento no banco de dados, evitando o armazenamento de senhas em texto plano.

VISÃO DE CASOS DE USO

Esta seção apresenta os casos de uso arquiteturalmente mais significativos da JoStore. O diagrama concentra o fluxo de pedidos, que envolve identificação do cliente, realização do pedido e solicitações de cancelamento ou troca. Ele representa o comportamento-alvo do sistema; a autenticação pela API será implementada em etapa posterior, substituindo o mecanismo temporário de acesso local usado no protótipo.

Diagrama de Casos de Uso de Pedidos

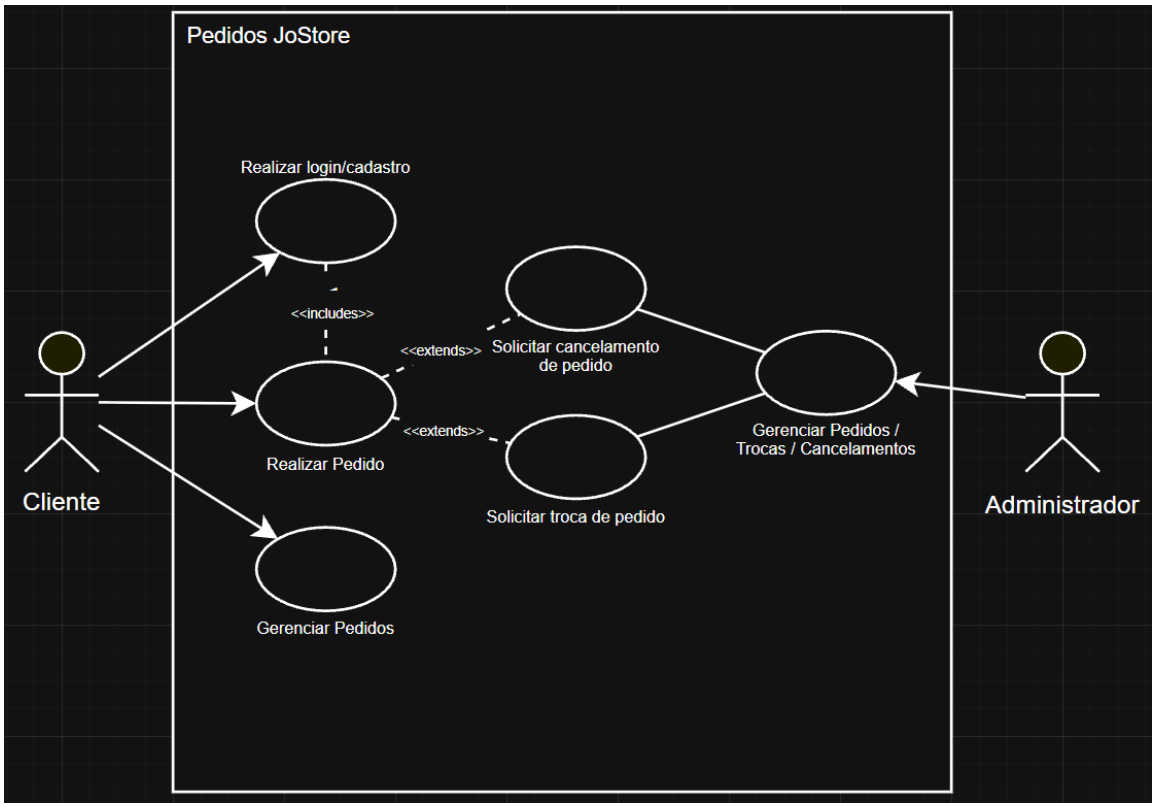


Figura 2 - Diagrama de casos de uso de pedidos

Descrição do Caso de Uso de Pedidos

Pedidos

1. Realizar Login/Cadastro: no fluxo-alvo, o cliente se identifica por meio de credenciais ou realiza seu cadastro. O sistema valida os dados e permite a continuidade do pedido. Na versão atual do protótipo, o acesso é mantido localmente; a autenticação da API permanece planejada.
2. **Realizar Pedido:** Este caso de uso tem início quando o cliente decide efetuar uma compra. Este caso de uso inclui obrigatoriamente o login/cadastro do cliente, já que não é possível realizar um pedido sem estar autenticado. O sistema registra o pedido com os itens selecionados e o disponibiliza para acompanhamento.
3. **Solicitar Cancelamento de Pedido:** Este caso de uso estende o Realizar Pedido e tem início quando o cliente deseja cancelar um pedido já efetuado. O sistema altera o status do pedido, sinalizando o cancelamento para o Administrador.
4. **Solicitar Troca de Pedido:** Este caso de uso também estende o Realizar Pedido e tem início quando o cliente deseja trocar um item recebido. O sistema altera o status do pedido conforme o fluxo de trocas definido, sinalizando a solicitação para o Administrador.
5. **Gerenciar/Acompanhar Meus Pedidos:** Este caso de uso tem início quando o cliente deseja consultar o andamento de seus próprios pedidos. O sistema exibe o status atual de cada pedido do cliente autenticado.



6. **Gerenciar Pedidos, Trocas e Cancelamentos:** Este caso de uso tem início quando o Administrador precisa consultar e tratar os pedidos, solicitações de troca e cancelamento de todos os clientes. O sistema permite ao Administrador alterar o status dos pedidos conforme o fluxo definido, aceitando ou negando solicitações de troca e processando cancelamentos.

VISÃO DE LÓGICA

Esta seção detalha a organização interna do sistema JoStore em camadas e pacotes, do ponto de vista de design, complementando a visão arquitetural apresentada anteriormente.

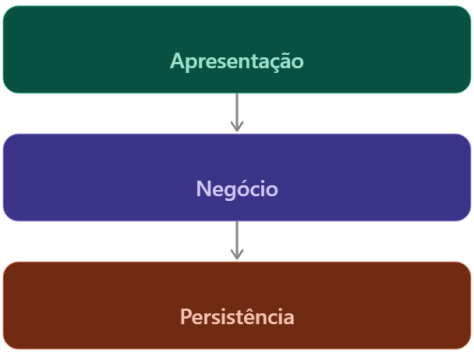


Figura 3 - Organização lógica em camadas

Apresentação: reúne o front-end React com TypeScript, organizado em páginas, componentes, estilos e serviços de comunicação com a API. Essa camada apresenta o catálogo, formulários de cadastro, perfil, carrinho e funcionalidades administrativas ao usuário.

Negócio: reúne os controllers e services do Spring Boot, responsáveis por receber requisições REST, aplicar regras de negócio e coordenar as operações do domínio.

Persistência: reúne as entidades JPA, repositories Spring Data e migrations Flyway que permitem consultar e persistir dados no PostgreSQL.

A **Erro! Fonte de referência não encontrada.**abaixo ilustra o diagrama de camadas com as t ecnologias utilizadas no desenvolvimento, já descritas na figura acima.

A figura abaixo ilustra o diagrama de camadas com as tecnologias utilizadas no desenvolvimento do JoStore, já descritas na figura anterior. A camada de apresentação depende do React com TypeScript, responsável pela construção da interface com o usuário e pela comunicação com a API REST. A camada de negócio depende do Spring Boot, responsável pela implementação dos Controllers e Services que concentram as regras de negócio da aplicação. A camada de persistência depende do PostgreSQL para o armazenamento dos dados, além de também se apoiar no Spring Boot, através do Spring Data JPA, para o acesso e a manipulação desses dados.

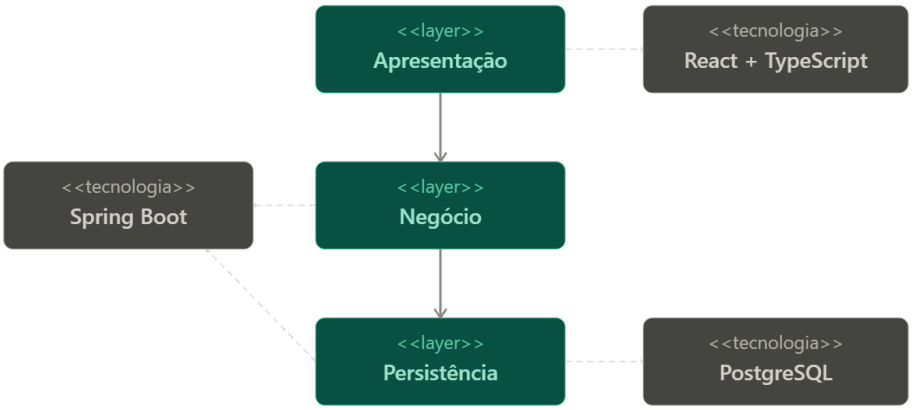


Figura 4 - Tecnologias associadas às camadas da JoStore

Camada de Apresentação

Nesta camada, temos a representação de pacotes da parte visual do sistema, onde contém os formulários para a inserção dos dados para posteriormente serem processados.

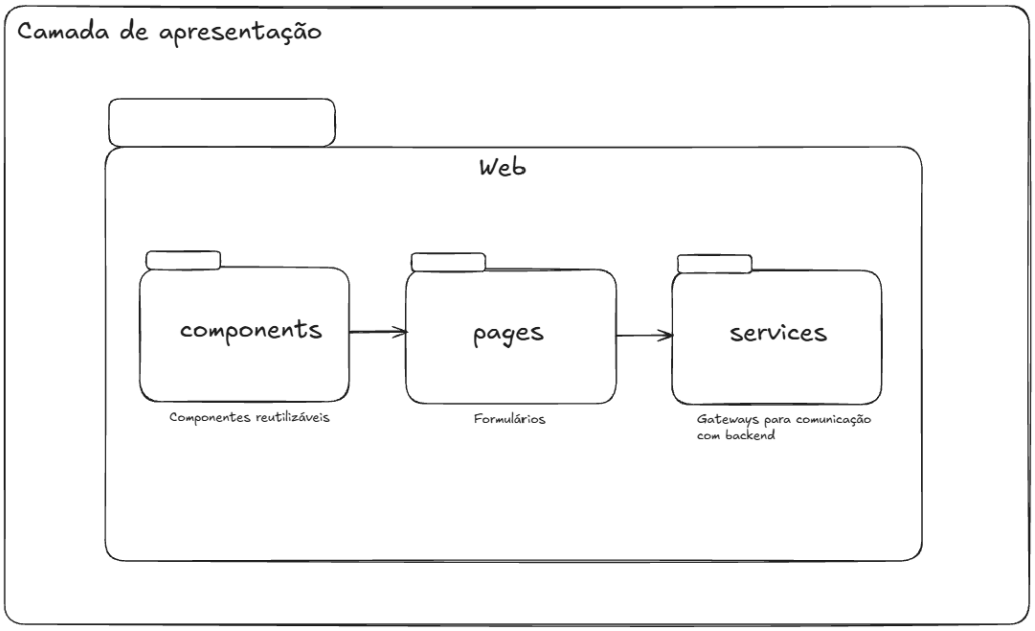


Figura 5 - Pacotes da camada de apresentação

Camada de Negócio

A camada de negócio do JoStore é implementada em Java com Spring Boot, sendo responsável por controlar a execução das funcionalidades do sistema. Ela é organizada nos pacotes controller, service, model, dto e exception, conforme apresentado no diagrama de pacotes abaixo.

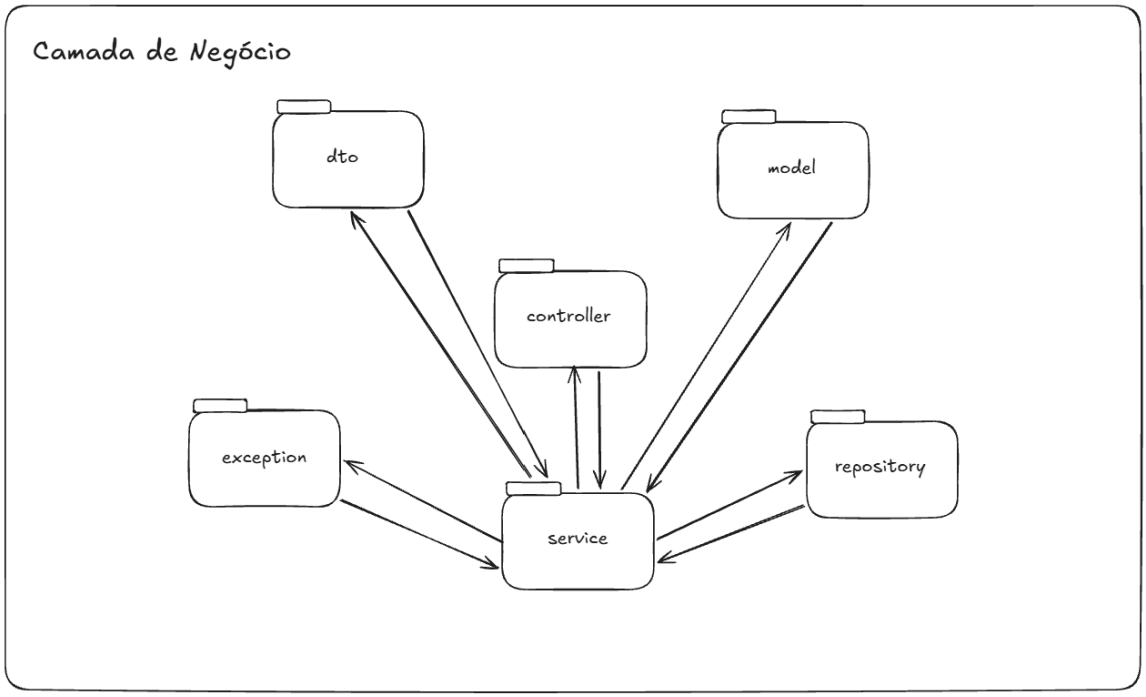


Figura 6 - Pacotes da camada de negócio

Pacote Controller

As figuras abaixo representam os diagramas de pacotes de controles do projeto JoStore implementadas em Java com Spring Boot.

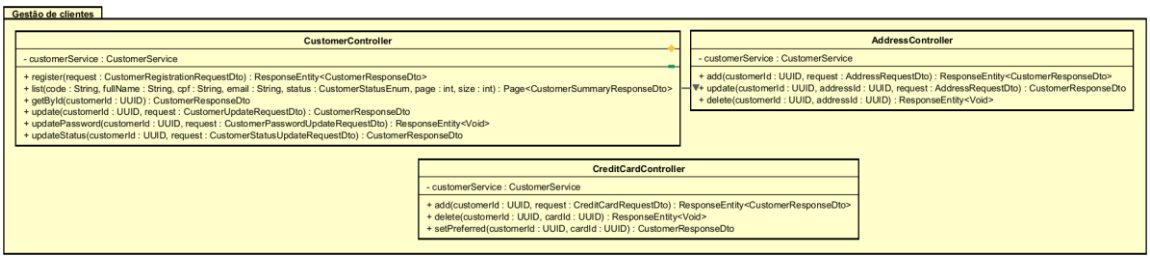


Figura 7 - Classes de controle do módulo de clientes

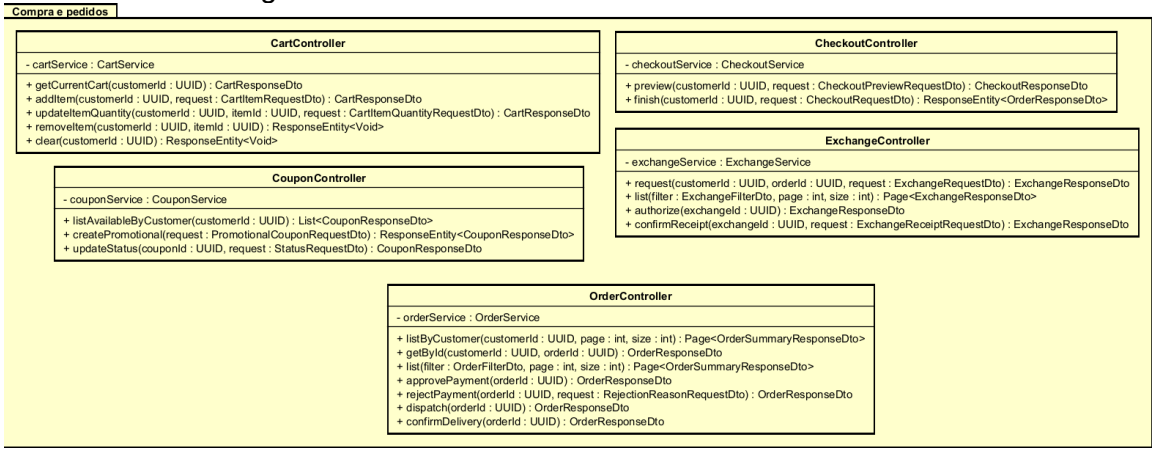


Figura 8 - Classes de controle complementares

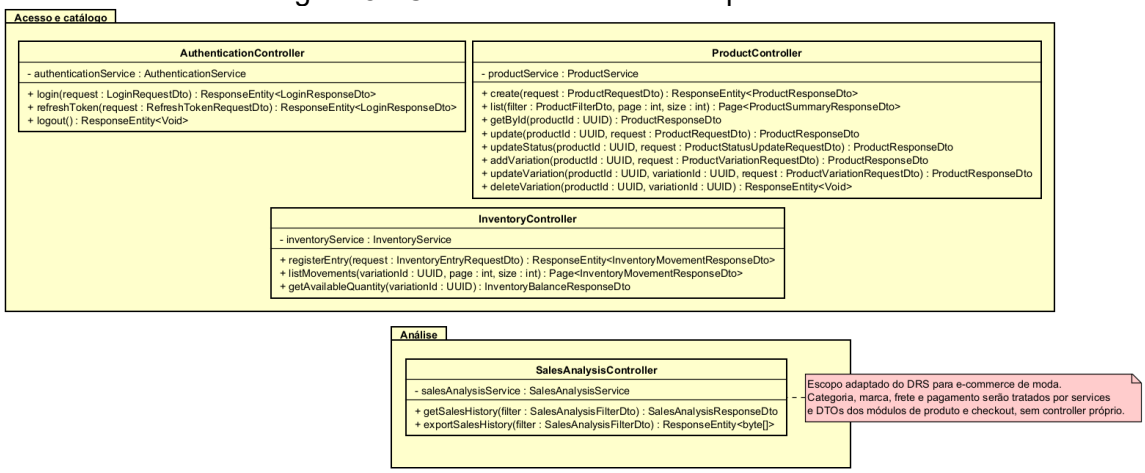


Figura 9 - Classes de modelo do módulo de clientes

Pacote Model

A figura abaixo ilustra as principais classes do modelo.



Figura 10 - Relacionamentos das classes do domínio

Camada de Persistência

Nesta camada está localizado o pacote repository, que contém as interfaces responsáveis pelo acesso e pela persistência dos dados no banco de dados relacional PostgreSQL. Os repositories utilizam o Spring Data JPA, que abstrai grande parte das operações de persistência, como inclusão, consulta, alteração e exclusão lógica dos dados. Internamente, o Spring Data JPA utiliza o Hibernate como implementação JPA para realizar o mapeamento objeto-relacional entre as entidades Java e as tabelas do banco de dados. Na figura abaixo contém os repositories relacionados a cliente.

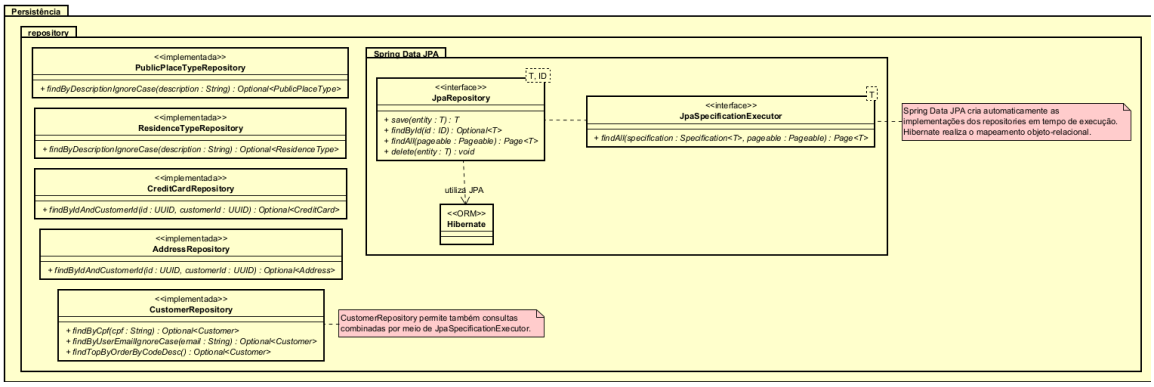


Figura 11 - Repositories do módulo de clientes

Realização dos Casos de Uso Significativos

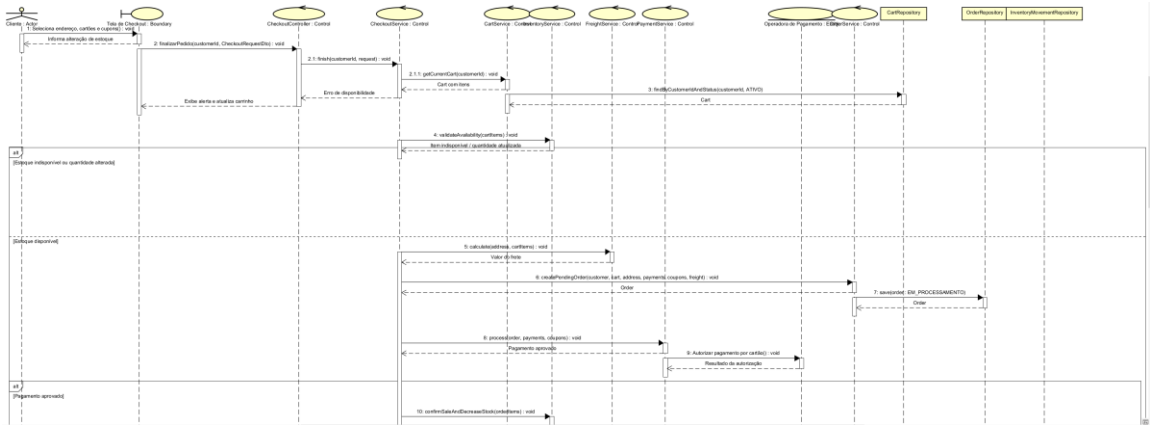


Figura 12 - Diagrama de sequência da realização de pedido

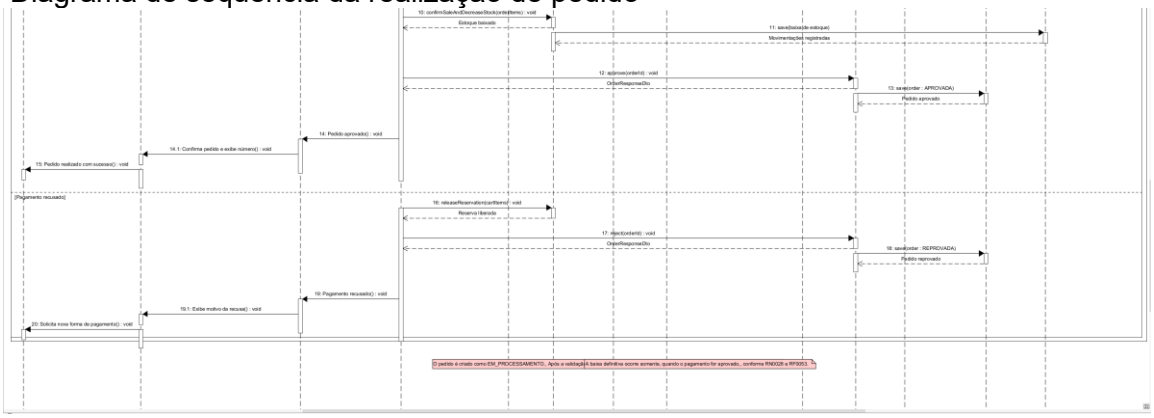


Figura 13 - Colaboração entre camadas no fluxo de pedido

VISÃO DE IMPLANTAÇÃO

Esta seção apresenta a configuração física e lógica utilizada para executar a JoStore no ambiente atual de desenvolvimento. A aplicação é acessada por navegadores web, tanto por clientes quanto por administradores.

O frontend foi desenvolvido em React com TypeScript e é executado pelo servidor de desenvolvimento Vite, na porta 5173. Ele se comunica por requisições HTTP com a API REST da JoStore, desenvolvida em Java 21 com Spring Boot e disponibilizada na porta 8080.

A API concentra as controllers, services, repositories, regras de negócio e o mapeamento objeto-relacional realizado por Spring Data JPA e Hibernate. A persistência é realizada no PostgreSQL 18, executado em container Docker na porta 5432, com volume persistente para armazenar os dados do banco. As migrations são controladas pelo Flyway.

No ambiente de produção, o servidor Vite será substituído por hospedagem estática ou servidor web, mantendo a comunicação entre frontend, API e banco de dados.

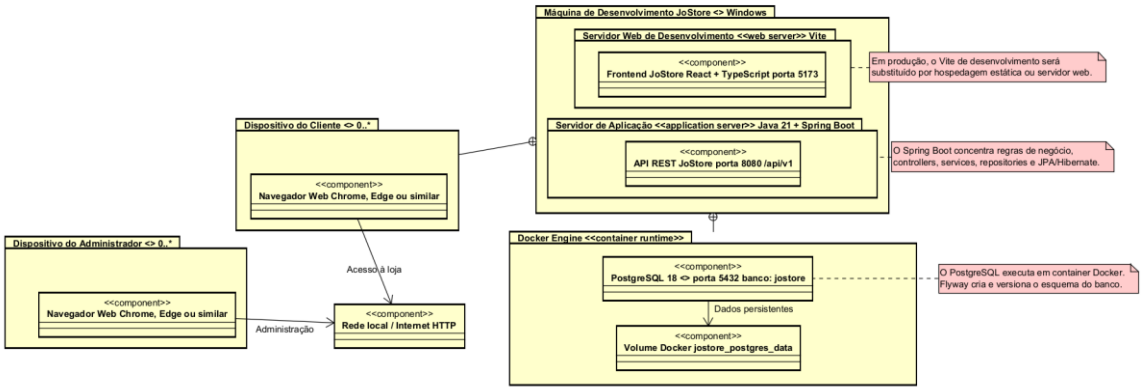


Figura 14 - Visão de implantação da JoStore

Na Figura acima observam-se os seguintes nós físicos:

- **Dispositivo do Cliente / Administrador:** computador ou dispositivo móvel utilizado para acessar a JoStore por meio de um navegador web. A interface é uma aplicação SPA desenvolvida em React e TypeScript.
- **Servidor Web de Desenvolvimento:** nó responsável por disponibilizar o frontend da JoStore. No ambiente atual, utiliza o Vite na porta 5173 para servir a aplicação React.
- **Servidor de Aplicação:** nó responsável por executar a API REST da JoStore, desenvolvida em Java 21 com Spring Boot. Nele estão as controllers, services, repositories, validações e regras de negócio. A API é disponibilizada na porta 8080.
- **Servidor de Banco de Dados:** nó responsável pelo banco central da aplicação. Utiliza PostgreSQL 18, executado em um container Docker na porta 5432. Os dados são mantidos em volume persistente do Docker.
- **Volume Persistente Docker:** armazenamento persistente associado ao container PostgreSQL, responsável por preservar as tabelas, registros e histórico de migrations mesmo quando o container é reiniciado.

No diagrama atual, o servidor web, servidor de aplicação, Docker e PostgreSQL estão na mesma máquina de desenvolvimento. Em uma futura implantação em produção, esses componentes poderão ser distribuídos em servidores independentes.



VISÃO DE IMPLEMENTAÇÃO

Esta visão descreve a organização do software da JoStore em módulos, camadas e pacotes de implementação. A estrutura adotada segue uma arquitetura MVC simplificada, separando responsabilidades de apresentação, controle, regras de negócio e persistência de dados.

O frontend e o backend são projetos independentes. O frontend é desenvolvido em React com TypeScript e é responsável pela interface do usuário. O backend é desenvolvido em Java com Spring Boot, seguindo a organização em camadas Controller, Service, Repository e Model, conforme detalhado na Visão de Lógica.

VISÃO DE DADOS

O mecanismo de persistência do JoStore utiliza o banco de dados relacional PostgreSQL 18, executado em contêiner Docker. O mapeamento objeto-relacional é realizado pelo Spring Data JPA, utilizando o Hibernate como implementação JPA. As alterações estruturais do banco são controladas por migrations do Flyway, enquanto as transações de negócio são delimitadas pelos serviços do Spring Framework.

As Figuras 15 e 16 apresentam, respectivamente, o modelo lógico completo planejado para a JoStore e o modelo físico atualmente implementado para o módulo de gestão de clientes. As entidades de catálogo, estoque, carrinho, pedidos, pagamentos, cupons e trocas guiam a evolução do banco nas próximas etapas, enquanto o modelo físico registra apenas as tabelas que já existem no PostgreSQL.

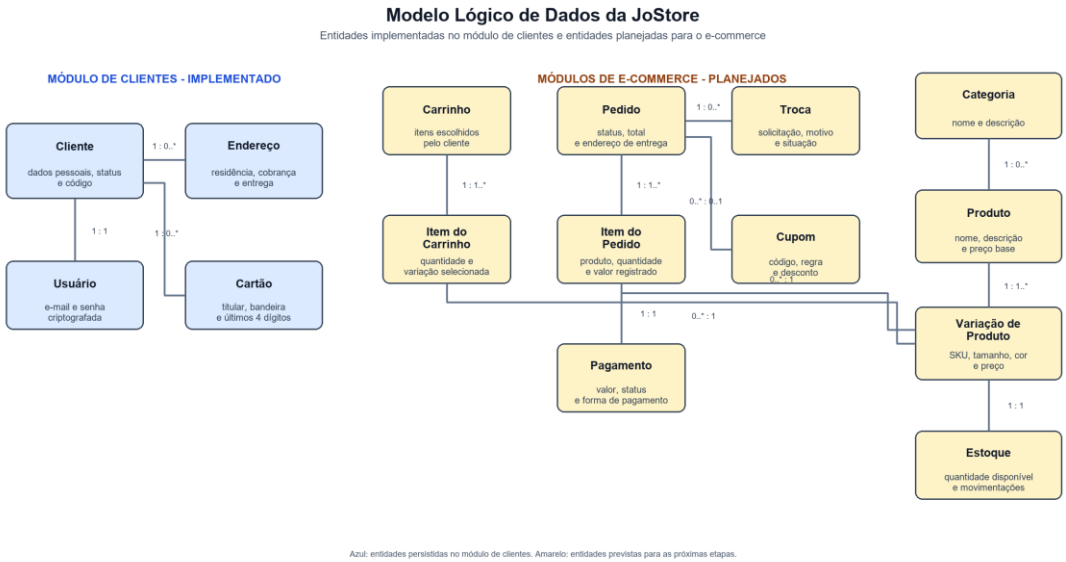


Figura 15 - Modelo lógico de dados planejado da JoStore



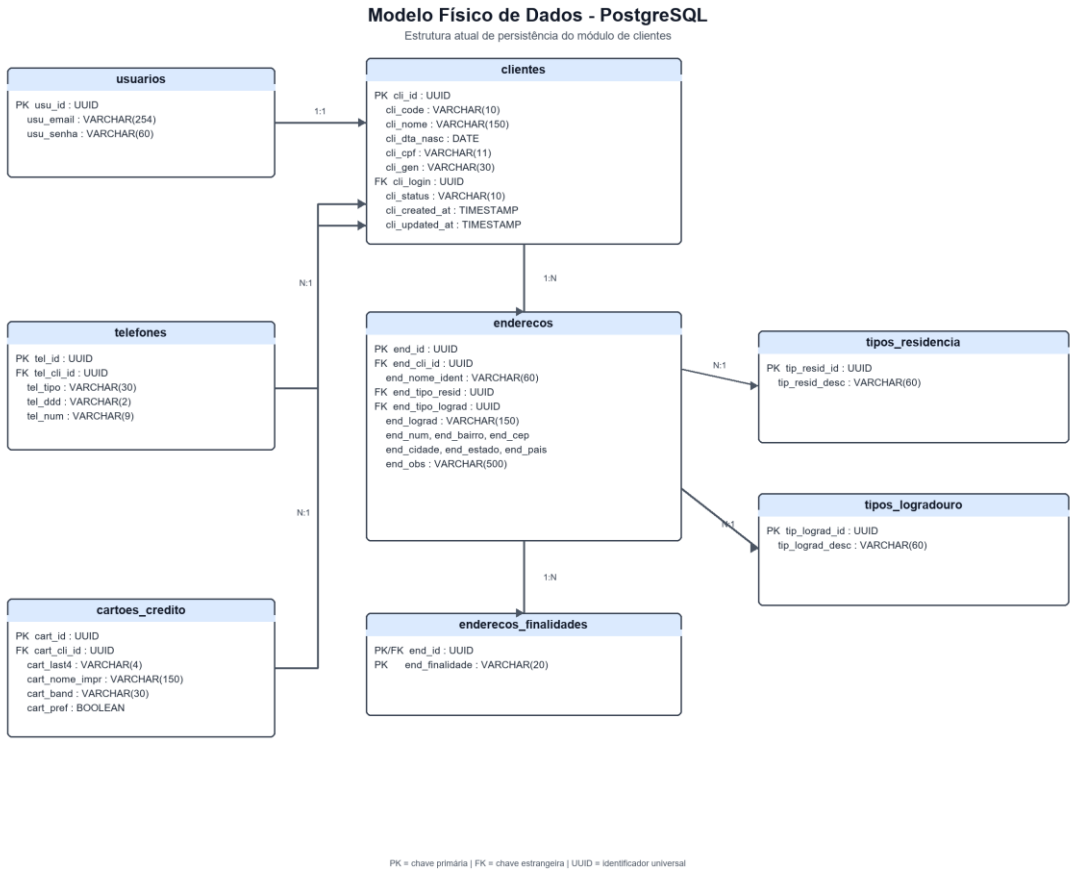


Figura 16 - Modelo físico de dados implementado para o módulo de clientes

A Tabela 2 apresenta o mapeamento entre as classes de modelo já implementadas no módulo de clientes e as entidades físicas persistidas no PostgreSQL.

Os identificadores das entidades persistentes utilizam UUID. As tabelas também impõem restrições relevantes ao domínio, como unicidade para CPF, e-mail e código do cliente, além da validação de formatos e valores permitidos para status, gênero, telefones, finalidades de endereço e bandeiras de cartão.

Os tipos de residência e de logradouro são mantidos em tabelas de domínio, permitindo que endereços façam referência a valores padronizados. As finalidades de um endereço são armazenadas na tabela associativa enderecos_finalidades, pois um mesmo endereço pode ser residencial, de cobrança e de entrega.

Por segurança, a senha do usuário é armazenada de forma criptografada. Os dados completos do cartão e o código de segurança não são persistidos: o sistema mantém apenas o titular, a bandeira, os quatro últimos dígitos e a indicação de cartão preferencial.

Tabela 2 - Mapeamento objeto-relacional do módulo de clientes

Classe de modelo	Entidade física
CUSTOMER	clientes
USER	usuarios
Phone	telefones
Address	enderecos
AddressPurposeEnum	enderecos_finalidades
CreditCard	cartoes_credito
ResidenceType	tipos_residencia
PublicPlaceType	tipos_logradouro



TAMANHO E PERFORMANCE

O JoStore é desenvolvido inicialmente como um e-commerce de pequeno porte e para fins acadêmicos. Para o piloto, estima-se até 200 contas cadastradas, cerca de 500 produtos e variações no catálogo, até 50 pedidos por dia e picos de aproximadamente 20 usuários simultâneos em campanhas e divulgações.

Os momentos de maior uso são esperados em campanhas, divulgações nas redes sociais e datas comerciais. Para esse cenário, as consultas administrativas de clientes utilizam paginação e filtros combináveis, e o banco de dados possui índices e restrições de unicidade para CPF, e-mail e código do cliente.

A infraestrutura atual é local, composta pela aplicação Spring Boot e pelo PostgreSQL em Docker. Caso o volume real ultrapasse essas estimativas, o dimensionamento de recursos, o monitoramento e as estratégias de cópia de segurança deverão ser revisados antes da implantação em produção.

QUALIDADE

A qualidade do JoStore está centrada na confiabilidade dos fluxos de venda e na proteção dos dados cadastrais dos clientes. As regras de negócio são aplicadas na camada de serviços, e os dados recebidos pela API são validados por DTOs e pelas restrições do banco de dados antes de serem persistidos.

A manutenção do sistema é favorecida pela arquitetura MVC e pela separação entre controllers, services, repositories, entidades, DTOs e tratamento centralizado de exceções. Essa organização reduz o acoplamento entre a interface web e a persistência e permite evoluir os módulos de catálogo, estoque, carrinho e pedidos sem concentrar regras em uma única camada.

Quanto à segurança, as senhas são criptografadas antes do armazenamento. O sistema não persiste o número completo ou o código de segurança de cartões. Os fluxos de interface do CRUD de clientes possuem testes automatizados com Cypress; testes de integração completos entre interface, API e banco serão ampliados nas próximas etapas.

Nas próximas etapas, a autenticação da API, o controle de autorização por perfil, a observabilidade da aplicação e a política de cópias de segurança devem complementar os mecanismos já implementados.

CRONOGRAMA MACRO

O cronograma a seguir organiza os principais marcos do desenvolvimento do JoStore. Os itens concluídos representam a base já construída; as etapas seguintes serão realizadas conforme o cronograma acadêmico e a evolução dos requisitos do e-commerce.

Marco	Semana / situação
Documento de Visão de Projeto e requisitos iniciais	Semanas 1 e 2 - Concluído
Protótipo da interface em React	Semanas 3 a 5 - Concluído



Modelagem do banco e CRUD de clientes	Semanas 6 a 10 - Concluído
Integração do front-end com a API de clientes	Semanas 9 e 10 - Concluído
Catálogo de produtos e gestão de estoque	Semanas 11 a 13 - Próxima etapa
Carrinho, checkout e fluxo de pedidos	Semanas 14 a 17 - Planejado
Trocas, cupons e relatórios administrativos	Semanas 18 e 19 - Planejado
Testes integrados, documentação e apresentação	Semanas 20 e 21 - Fase final

Observação: os marcos futuros podem ser ajustados de acordo com as entregas definidas pela disciplina e com a validação dos requisitos junto à proprietária da loja.



REFERÊNCIAS

JoStore. Documento de Requisitos do Sistema (DRS_LES_2_2026).
JoStore. Contrato OpenAPI do módulo de clientes (customer-api.yaml).

Spring. Spring Boot Reference Documentation. Disponível em: https://docs.spring.io/spring-boot/

PostgreSQL Global Development Group. PostgreSQL Documentation. Disponível em: https://www.postgresql.org/docs/

Flyway. Documentation. Disponível em: https://documentation.red-gate.com/flyway

Cypress. Documentation. Disponível em: https://docs.cypress.io/

Object Management Group. Unified Modeling Language (UML). Disponível em: https://www.omg.org/spec/UML/